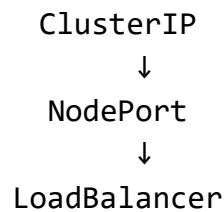


Kubernetes Services, NodePort & LoadBalancer on EKS

This SOP documents everything we learned from creating Services, troubleshooting networking, exposing applications, and understanding traffic flow inside EKS.

Objective

Learn how applications are exposed in Kubernetes using and understand how traffic reaches Pods.



Environment

Existing Resources

EKS Cluster

mani-eks-cluster

Node Group

my-eks-node-group

Worker Nodes

172.31.17.46

172.31.40.167

Deployment

nginx-deployment

Phase 4A - Service Fundamentals

Verify Deployment

Check deployment exists.

kubectl get deployments

Expected:

NAME	READY
nginx-deployment	1/1

Verify Pod

```
kubectl get pods -o wide
```

Example:

NAME	IP
nginx-deployment-b6485fcbb-sfdm6	172.31.39.92

Why We Need Services

Pods are temporary.

Example:

Current Pod

172.31.39.92

Delete pod:

```
kubectl delete pod <pod-name>
```

New pod:

172.31.21.188

Problem:

Pod IP changes

Applications cannot depend on Pod IPs.

Solution

Create a Service.

Service provides:

Stable IP

Stable Name

Traffic Routing

Create ClusterIP Service

File:

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: nginx-service
```

```
spec:
```

```
  selector:
```

```
    app: nginx
```

```
  ports:
```

```
    - port: 80
```

```
      targetPort: 80
```

```
  type: ClusterIP
```

Apply:

```
kubectl apply -f nginx-service.yaml
```

Verify Service

```
kubectl get svc
```

Example:

NAME	TYPE	CLUSTER-IP
nginx-service	ClusterIP	10.100.89.239

Service Components

ClusterIP

10.100.89.239

Belongs to:

Service

Used internally.

Selector

Service finds Pods using:

```
selector:  
  app: nginx
```

Verify Labels

```
kubectl get pods --show-labels
```

Example:

```
app=nginx
```

Service matches pods using this label.

Verify Endpoints

```
kubectl get endpoints
```

Example:

```
nginx-service    172.31.39.92:80
```

Meaning:

Service

↓

Pod

```
172.31.39.92
```

Internal Connectivity Test

Create temporary troubleshooting pod.

```
kubectl run test-pod `   
  --image=busybox `   
  --restart=Never `   
  -it --rm -- sh
```

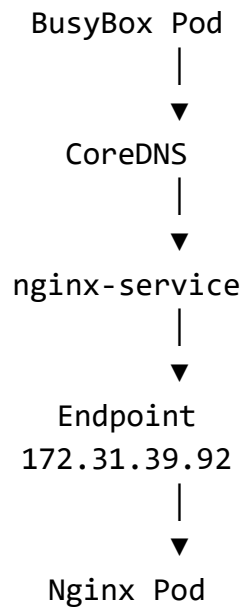
Inside:

```
wget -qO- http://nginx-service
```

Expected:

```
Welcome to nginx!
```

Traffic Flow



Phase 4B - NodePort Service

Goal: Expose application outside Kubernetes cluster.

Create NodePort Service

File:

apiVersion: v1

kind: Service

metadata:

name: nginx-nodeport

spec:

type: NodePort

selector:

app: nginx

ports:

- port: 80

targetPort: 80

nodePort: 30080

Apply:

```
kubectl apply -f nginx-nodeport.yaml
```

Verify

```
kubectl get svc
```

Example:

```
nginx-nodeport
```

```
80:30080/TCP
```

Understanding NodePort

Kubernetes opens:

```
Port 30080
```

on every worker node.

Worker Nodes

```
172.31.17.46
```

```
172.31.40.167
```

Verify NodePort Internally

BusyBox:

```
wget -qO- http://172.31.40.167:30080
```

Expected:

```
Welcome to nginx!
```

Security Group Requirement

NodePort traffic must be allowed.

Worker Node SG:

```
sg-0c023fc21e49937a6
```

Add:

TCP
30080
Source:
Windows EC2 SG

Test From Windows EC2

Successful test:

curl <http://172.31.40.167:30080>

Response:

Welcome to nginx!

Important Discovery

Public IP failed:

curl <http://35.154.16.83:30080>

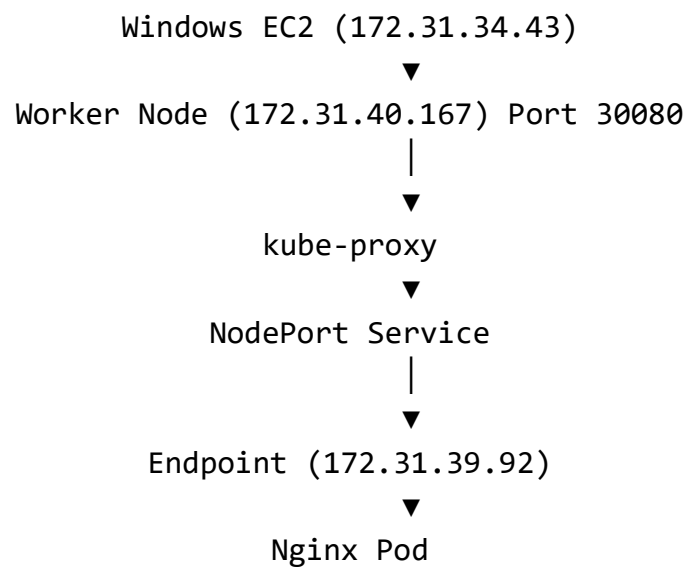
Private IP worked:

curl <http://172.31.40.167:30080>

Reason:

Same VPC and Use Private IP

NodePort Traffic Flow



Self-Healing Demonstration

Delete pod:

```
kubectl delete pod <pod-name>
```

Observe New Pod

Old:

172.31.39.92

New:

172.31.21.188

Verify Endpoint Updated

```
kubectl get endpoints nginx-nodeport
```

Example:

172.31.21.188:80

Kubernetes automatically updates routing.

No Service changes required.

Key Learning

Applications should never use:

Pod IP

Use:

Service

instead.

Phase 5 - LoadBalancer Service

Goal: Expose application using AWS Load Balancer.

Create LoadBalancer Service

File:


```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: nginx-loadbalancer
```

```
spec:
```

```
  type: LoadBalancer
```

```
  selector:
```

```
    app: nginx
```

```
  ports:
```

```
    - port: 80
```

```
      targetPort: 80
```

Apply:

```
kubectl apply -f nginx-loadbalancer.yaml
```

[Verify Service](#)

```
kubectl get svc
```

Initially:

```
EXTERNAL-IP - pending
```

[Wait For AWS](#)

AWS automatically creates:

Classic Load Balancer

Example:

```
a706fa3320d234387a4ad5efb731fff8-920586353.ap-south-1.elb.amazonaws.com
```

[Verify Details](#)

```
kubectl describe svc nginx-loadbalancer
```

What AWS Created

Behind the scenes:

Elastic Load Balancer

visible under:

EC2



Load Balancers

Test In Browser

Open:

<http://<elb-dns>>

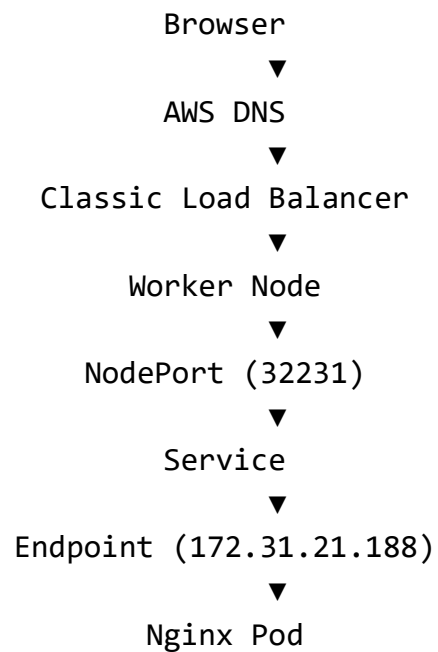
Example:

<http://a706fa3320d234387a4ad5efb731fff8-920586353.ap-south-1.elb.amazonaws.com>

Expected:

Welcome to nginx!

LoadBalancer Traffic Flow



Service Type Comparison

Type	Accessible From	Example
ClusterIP	Inside Cluster Only	10.100.89.239
NodePort	Node IP + Port	172.31.40.167:30080
LoadBalancer	Internet	ELB DNS

Most Important Learning From Phase 4 & 5

Never think:

User



Pod

Think:

User



Load Balancer



Service



Endpoint



Pod

The Service is the stable identity.

Pods come and go.

Services remain.

That single concept explains why Kubernetes networking works and why deleting a Pod never broke your application. 🚀